HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN 1

BÁO CÁO GIỮA KỲ

Xây dựng Chatbot hỏi đáp về lịch sử Việt Nam sử dụng GraphRAG

Lớp: E22HTTT

Tên: Nguyễn Đức Anh

ID: B22DCDT017

Người hướng dẫn: Kim Ngọc Bách

Hà Nội – 2026

BÁO CÁO TUẦN 3

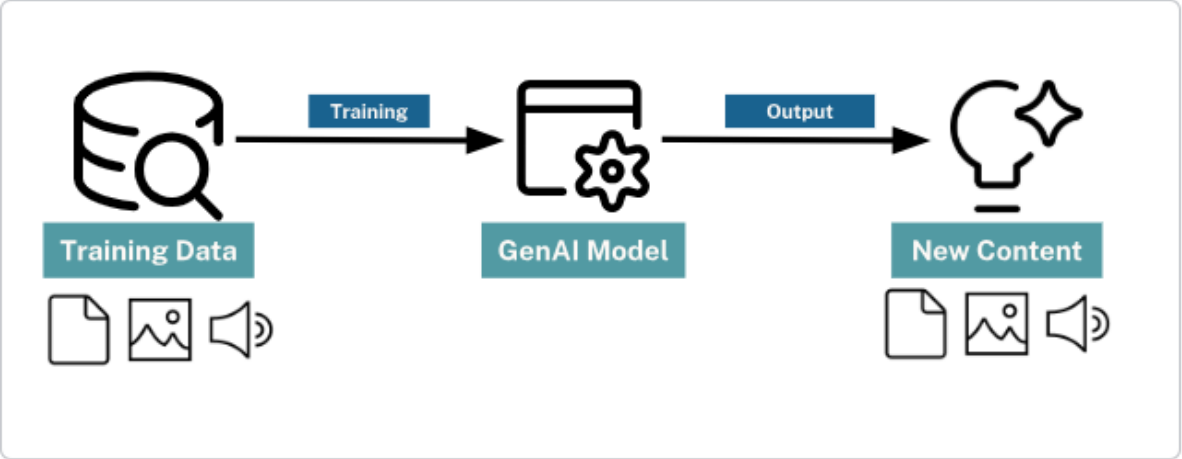
Neo4j & GenerativeAI Fundamentals

- What Generative AI is and how it works.

Generative AI (or GenAI) refers to artificial intelligence systems designed to create new content that resembles human-made data. The data could be text, images, audio, or code.

Generative AI is widely used in applications such as chatbots, content creation, image synthesis, and code generation.

These models, like GPT (for text) or DALL-E (for images), are trained on large datasets and use patterns learned from this data to generate new output.



- What Large Language Models (LLMs) are and how they differ from other AI models.

LLMs are a type of generative AI model designed to understand and generate human-like text.

These models are trained on vast amounts of text data and can perform various tasks, including answering questions, summarizing data, and analyzing text.

The LLM provides the most likely response based on the patterns it has learned from its training data.

- The limitations of Generative AI models, including *hallucination* and *access to data*.

Access to Data

The sources for this training data are often the internet, books, and other publicly available text. The data could be of questionable quality or even incorrect.

Training happens at a point in time, the data is static, and may not reflect the current state of the world or include any private information.

When prompted to provide a response, relating to new or data not in the training set, the LLM may provide a response that is not accurate.

Accuracy

LLMs are designed to create human-like text and are often fine-tuned to be as helpful as possible, even if that means occasionally generating misleading or baseless content, a phenomenon known as hallucination.

Temperature

A higher temperature may be appropriate when configuring an LLM to respond with more diverse and creative outputs, a lower temperature required when responses should be consistent and precise.

Transparency

GenAI Models are often considered "black boxes" due to the difficulty deciphering their decision-making processes.

The LLM is also unable to provide the sources for its output or explain its reasoning.

- How providing context can improve the responses from Generative AI models.

You can improve the accuracy of responses from LLMs by providing *context* in your prompts.

The context could include relevant information, data, or details that help the model generate more accurate and relevant responses.

Avoiding hallucination

Providing context can help minimize hallucinations by anchoring the model's response to the facts and details you supply.

Access to data

LLMs have a fixed knowledge cutoff and cannot access real-time or proprietary data unless it is provided in the prompt.

If you need the model to answer questions about recent events or organization-specific information, you must supply that data as part of your prompt. This ensures that the model's responses are up-to-date and relevant to your particular use case.

You could also provide statistics or data points in the prompt to help the model include useful facts in its response.

Supplying context

Supplying context in your prompts helps LLMs generate more accurate, relevant, and trustworthy responses by reducing hallucinations and compensating for the lack of access to data.

Retrieval Augmented Generation (RAG)

- What Retrieval Augmented Generation (RAG) is and how you can use it to improve GenerativeAI model responses.

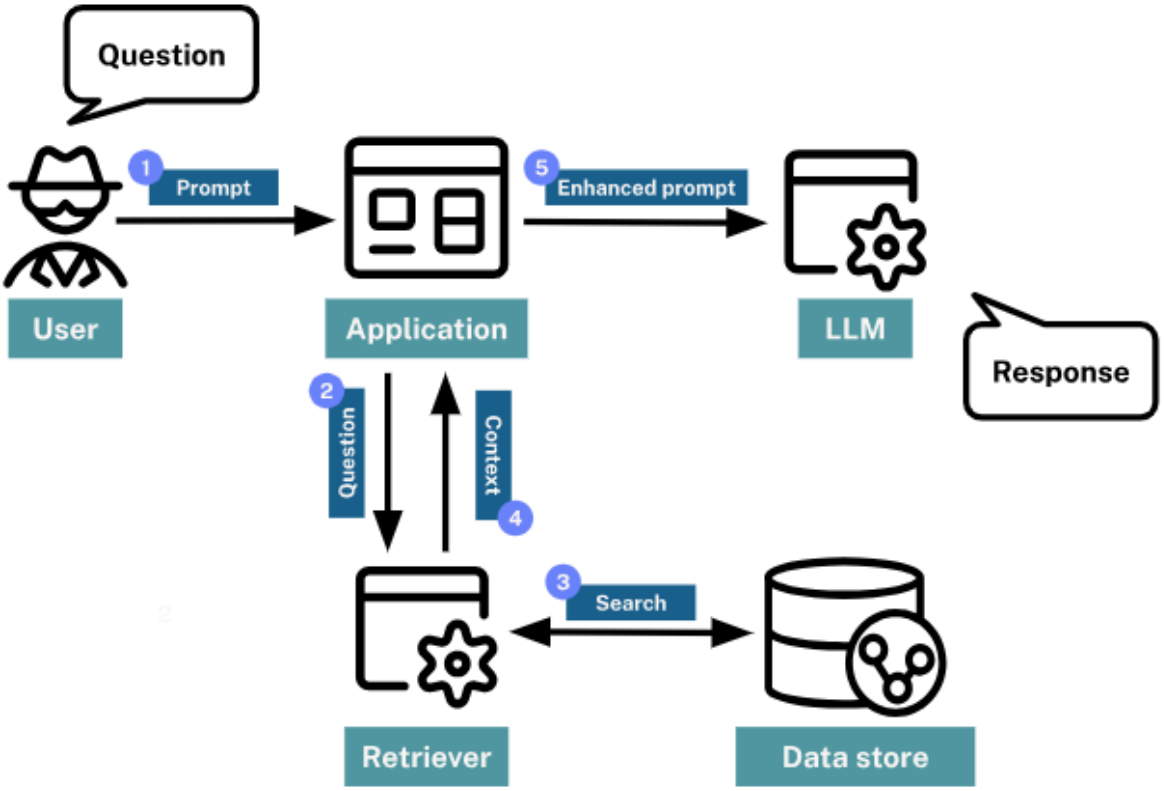
Retrieval-Augmented Generation (RAG) is an approach that enhances the responses of LLMs by providing them with relevant, up-to-date information retrieved from external sources.

RAG helps generate more accurate and tailored answers, especially when the required information is not present in the model's training data.

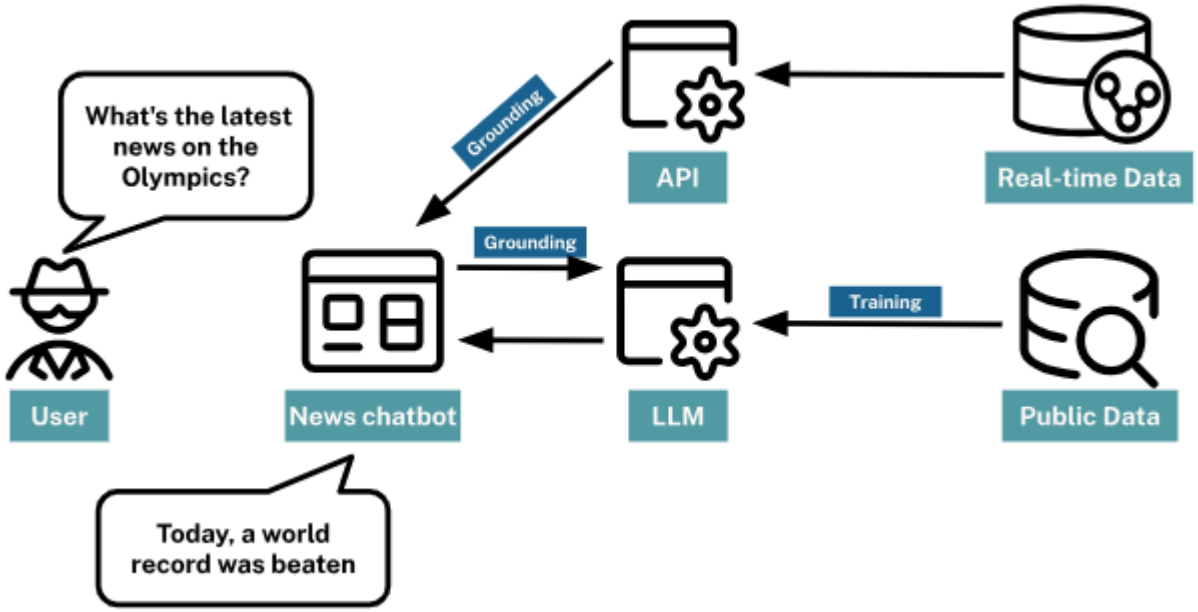
The Retrieval-Augmented Generation (RAG) Process

The RAG process typically involves three main steps:

- Understanding the User Query
  The system first interprets the user's input or question to determine what information is needed.
- Information Retrieval
  A *retriever* searches external data sources (such as documents, databases, or knowledge graphs) to find relevant information based on the user's query.
- Response Generation
  The retrieved information is inserted into the prompt, and the language model uses this context to generate a more accurate and relevant response.



If building a chatbot for a news agency, you could use RAG to pull real-time headlines or results from a news API.



Retrievers

The retriever is a key component of the RAG process. A retriever is responsible for searching and retrieving relevant information from external data sources based on the user's query.

A retriever typically takes an unstructured input (like a question or prompt) and searches for structured data that can provide context or answers.

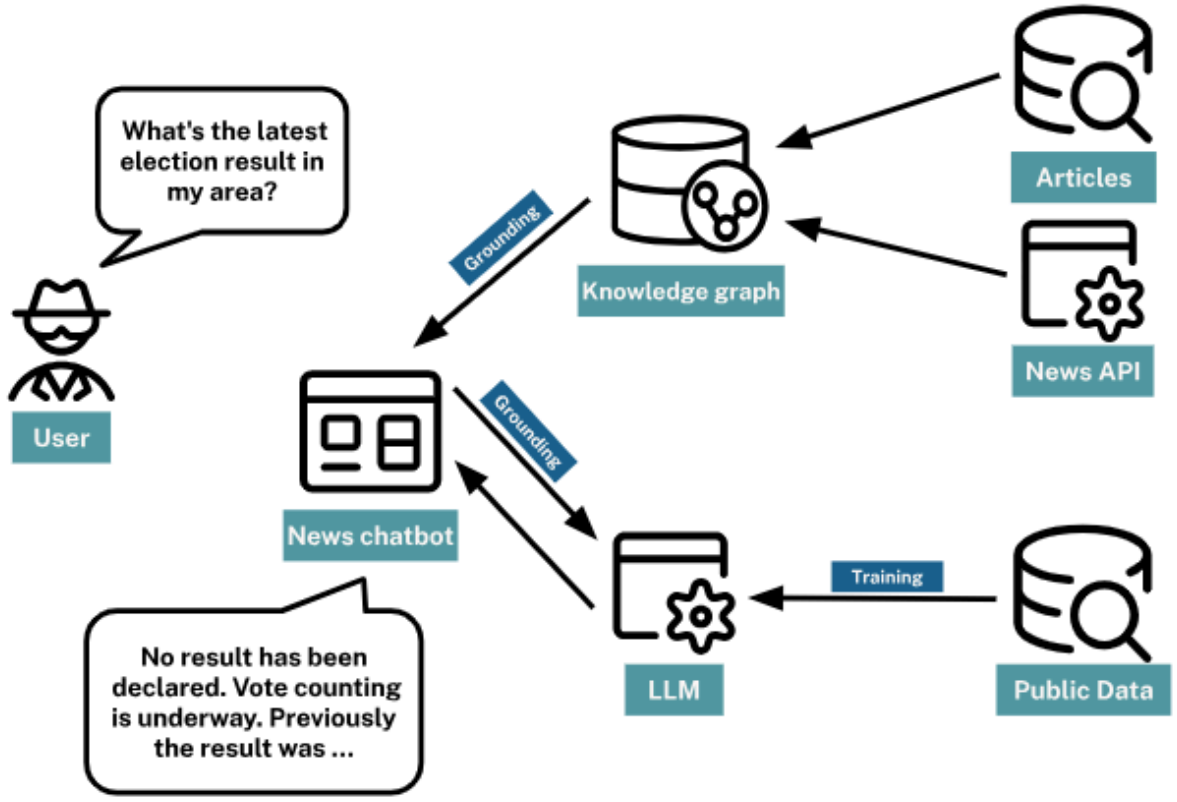
Data sources

The data sources used in the RAG process can vary widely, depending on the application and the type of information needed. Common data sources include:

- Documents
  Textual data sources, such as articles, reports, or manuals, that can be searched for relevant information.
- APIs
  External services that can provide real-time data or specific information based on user queries.
- Knowledge Graphs
  Graph-based representations of information that can provide context and relationships between entities.

The news agency chatbot could use the following data sources:

- A news API to retrieve the latest articles or headlines.
- A knowledge graph to understand the relationships between different news topics, such as how they relate to each other or their historical context. This would help the chatbot provide more in-depth and contextual responses.
- A document database to store and retrieve articles, reports, or other textual data that can be used to answer user queries.



- How vectors and embeddings work, and how they can be used in RAG to find relevant information.

Semantic Search

Semantic search aims to understand search phrases' intent and contextual meaning, rather than focusing on individual keywords.

Traditional keyword search often depends on exact-match keywords or proximity-based algorithms that find similar words.

The results are tailored based on the term and the perceived intent.

Vectors

You can represent data as vectors to perform semantic search.

Vectors are simply a list of numbers. For example, the vector `[1, 2, 3]` is a list of three numbers and could represent a point in three-dimensional space.

You can use vectors to represent many different types of data, including text, images, and audio.

The number of dimensions in a vector is called the dimensionality of the vector. A vector with three numbers has a dimensionality of 3. Higher dimensionality captures more fine-grained meaning but is more expensive computationally and similarly, lower dimensionality is faster and cheaper to compute, but offers less nuance.

Embeddings

When referring to vectors in the context of machine learning and NLP, the term "embedding" is typically used. Embeddings are numerical translations of data objects, for example images, text, or audio, represented as vectors. This way, LLM algorithms will be able to compare two different text paragraphs by comparing their numerical representations.

Each dimension in a vector can represent a particular semantic aspect of the word or phrase. When multiple dimensions are combined, they can convey the overall meaning of the word or phrase.

When applied in a search context, the vector for "apple" can be compared to the vectors for other words or phrases to determine the most relevant results.

You can create embeddings in various ways, but one of the most common methods is to use an embedding model.

While it is possible to create embeddings for individual words, embedding entire sentences or paragraphs is more common, as the meaning of a word can change based on its context.

For example, the word *bank* will have a different vector in *river bank* than in *savings bank*.

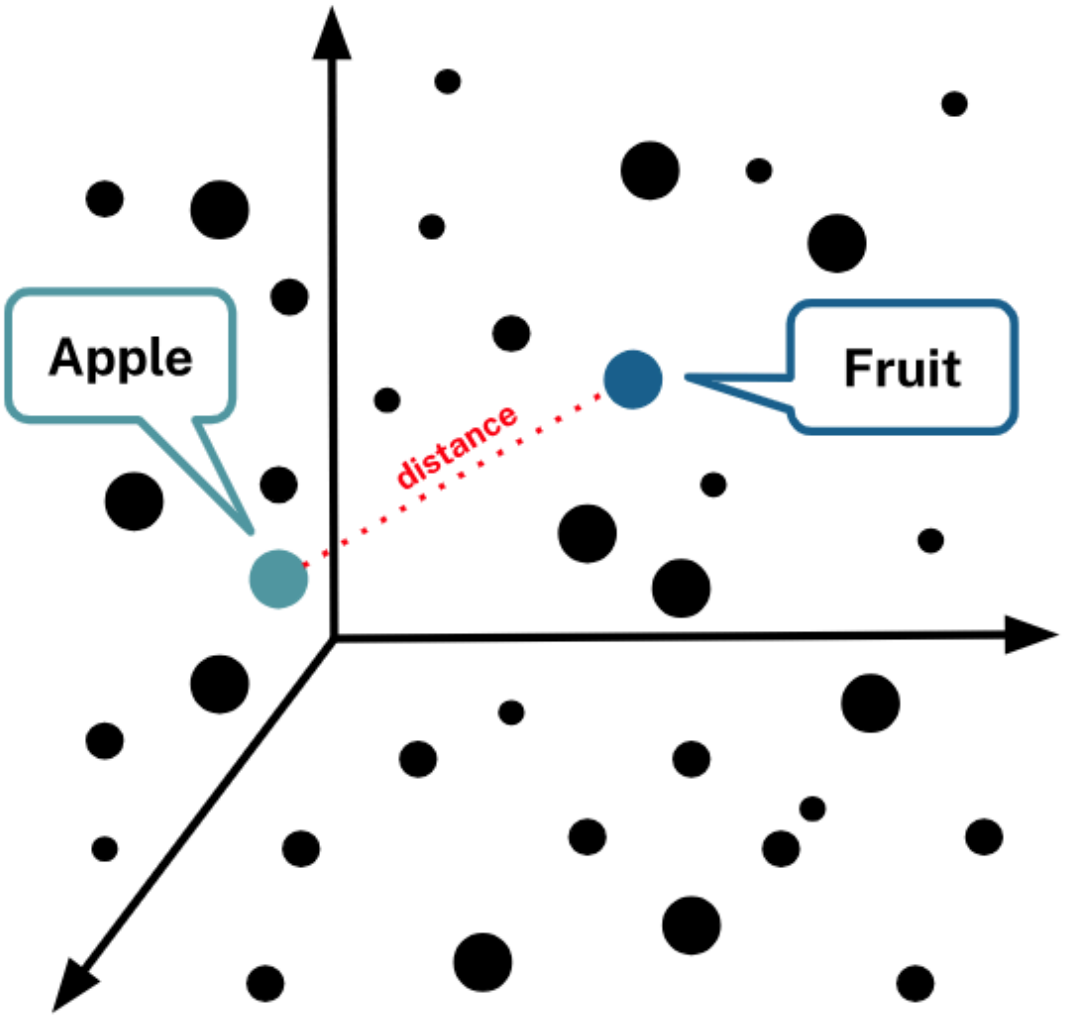
Semantic search systems can use these contextual embeddings to understand user intent.

Embeddings can represent more than just text. They can also represent entire documents, images, audio, or other data types.

How are vectors used in semantic search?

You can use the *distance* or *angle* between vectors to gauge the semantic similarity between words or phrases.

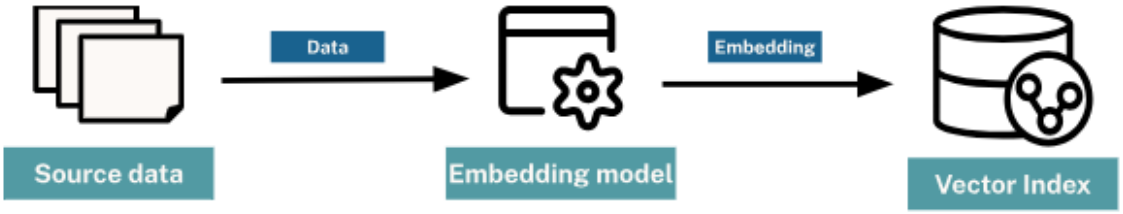
Words with similar meanings or contexts will have vectors that are close together, while unrelated words will be farther apart.



RAG

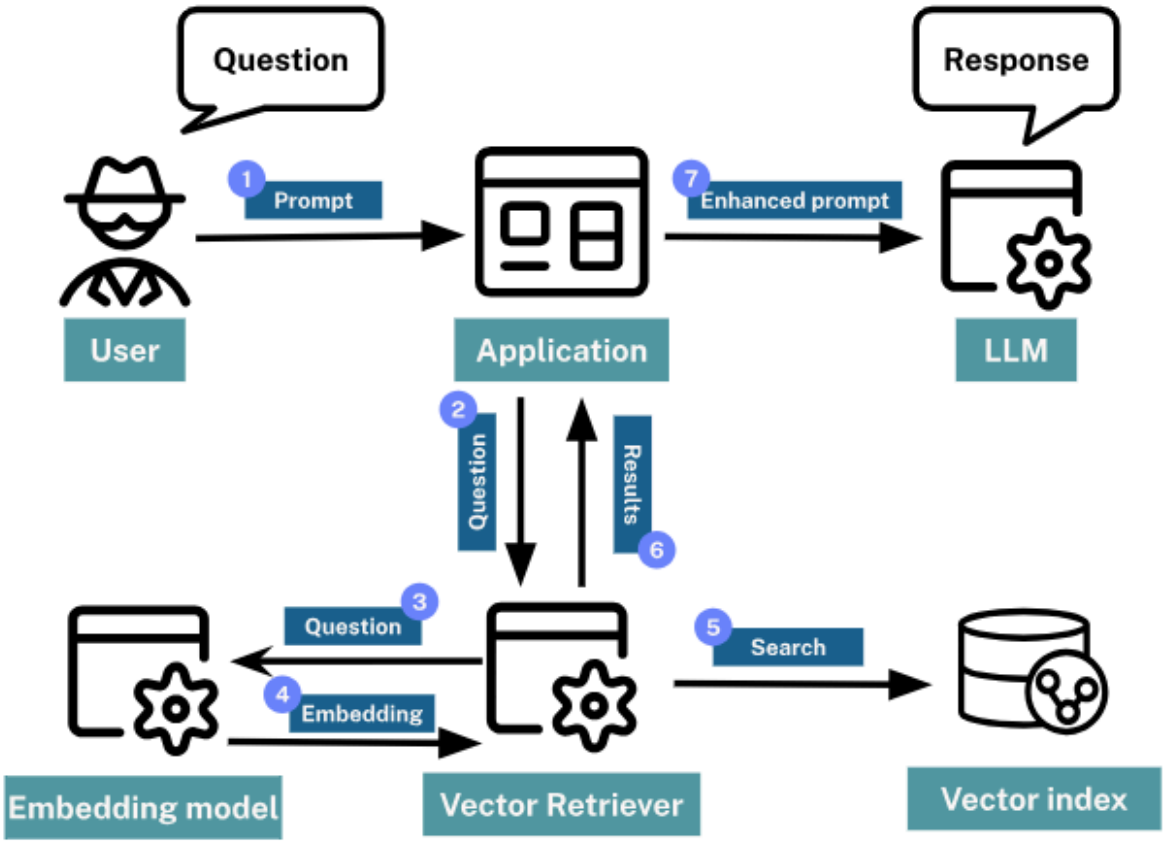
Semantic search is employed in vector-based RAG to find contextually relevant results for a user's question.

An embedding model is used to create a vector representation of the source data.



When a user submits a question, the system:

- Creates an embedding of the question.
- Compares the question vector to the vectors of the indexed data.
- The results are scored based on their similarity.
- The most relevant results are used as context for the LLM.



- How to use a vector indexes in Neo4j and when they are useful for finding context for Generative AI applications.

Vectors work well for:

- Contextual or Meaning Based Questions
- Fuzzy or Vague queries
- Broad or Open-Ended questions
- Complex queries with multiple concepts

Vectors are ineffective for:

- Highly Specific or Fact-Based Questions
- Numerical or Exact-Match Queries
- Boolean or Logical Queries
- Ambiguous or Unclear Queries without Context
- Specialized Knowledge

- About GraphRAG techniques, and how they can be used to enhance information retrieval.

Benefits of GraphRAG:

- Richer Context
  Graphs capture relationships between entities, enabling retrieval of more relevant and connected information.
- Improved Accuracy
  By combining vector similarity with graph traversal, results are more precise and context-aware.
- Explainability
  Graphs provide clear paths and connections, making it easier to understand why certain results were retrieved.
- Flexible Queries:
  GraphRAG supports complex queries, such as combining full-text, vector, and text-to-cypher searches.
- Enhanced Reasoning
  Graphs enable reasoning over data, supporting advanced use cases like recommendations and knowledge discovery.

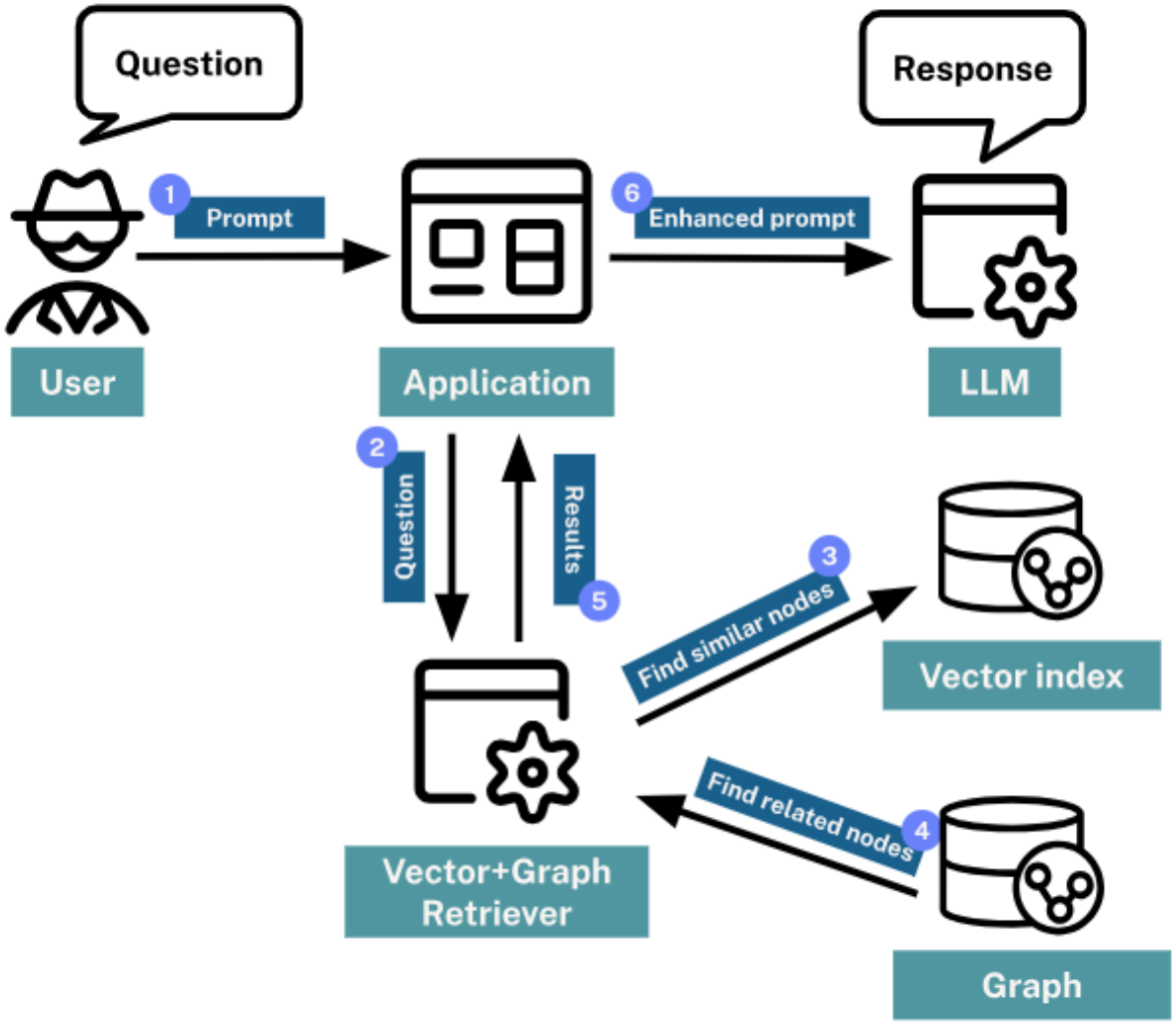
Graph-Enhanced Vector Search

A common approach to GraphRAG is to use a combination of vector search and graph traversal.

This allows for the retrieval of relevant documents based on semantic similarity, followed by a graph traversal to find related entities or concepts.

The high-level process is as follows:

1. A user submits a query.
2. The system uses a vector search to find nodes similar to the users query.
3. The graph is then traversed to find related nodes or entities.
4. The entities and relationships are added to the context for the LLM.
5. The related data could also be scored based on its relevance to the user query.



Full Text Search

Full text search is another powerful technique that can be combined with graph-enhanced search to further improve information retrieval.

While vector search excels at finding semantically similar content, full text search allows users to match specific keywords or phrases within documents or nodes.

If the user is looking for a movie or actor by name, full text search can quickly locate those entities based on exact text matches.

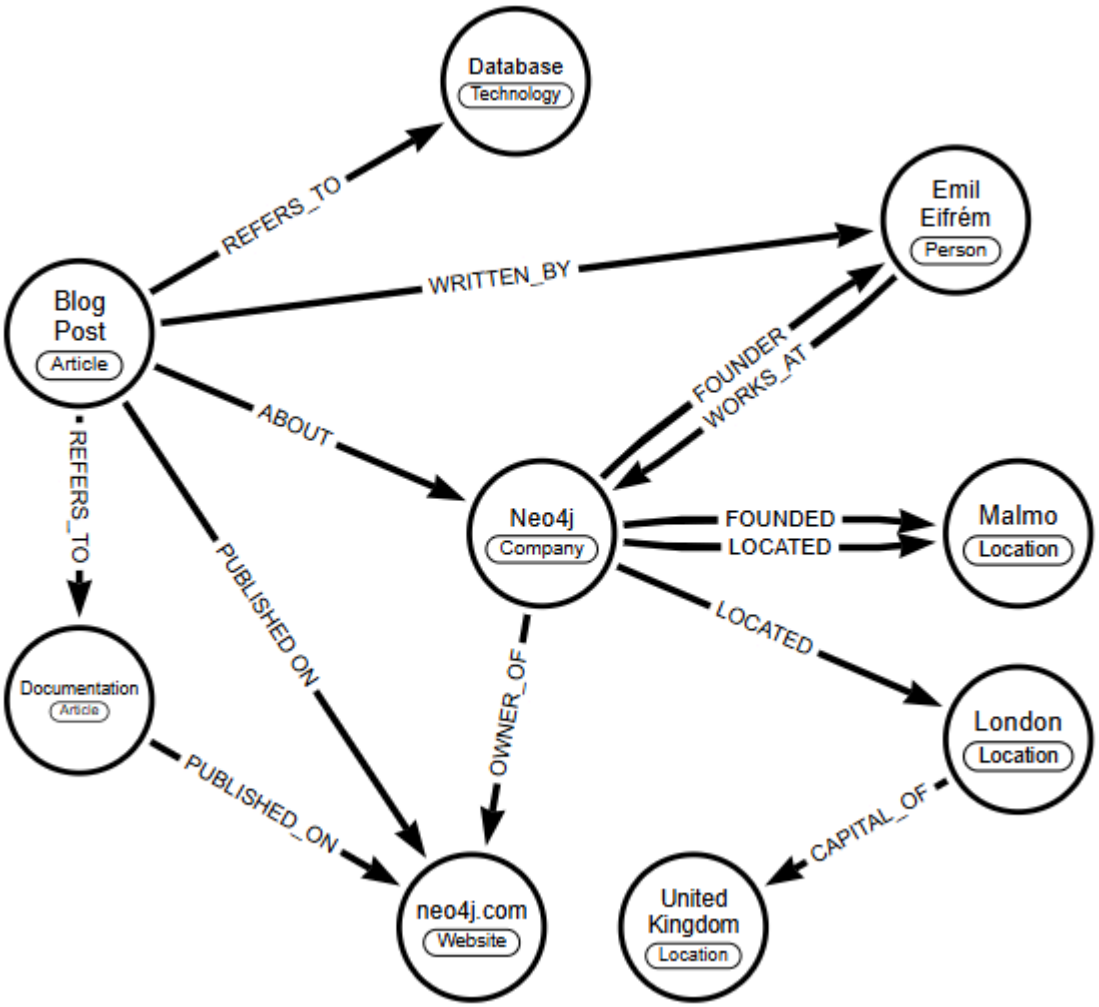
Full text search can be used as a replacement for or in conjunction with vector search.

When used in conjunction with vector search, full text search can refine results by filtering out irrelevant content based on specific keywords or phrases.

Knowledge Graphs

Knowledge graphs provide a structured way to represent entities, their attributes, and their relationships, allowing for a comprehensive and interconnected understanding of the information.

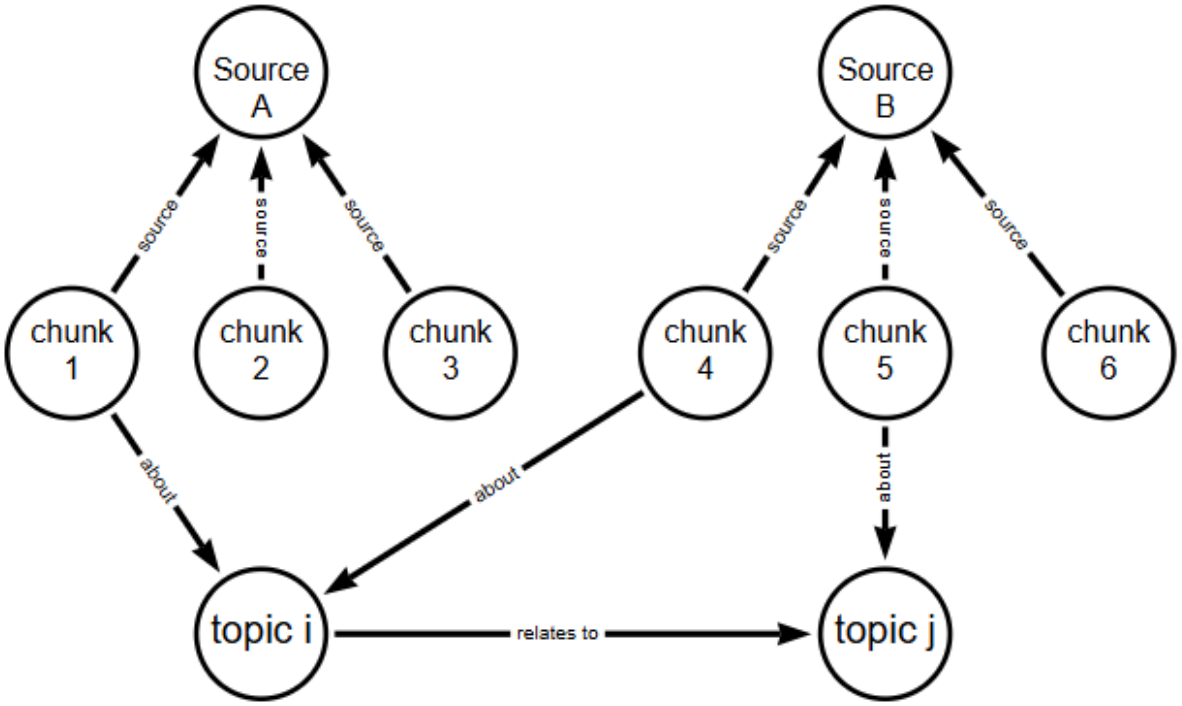
Knowledge graphs are useful for Generative AI applications because they provide structured, interconnected data that enhances context, reasoning, and accuracy in generated responses.



Knowledge graphs can break down sources of information and integrate them, allowing you to see the relationships between the data.

This integration from diverse sources gives knowledge graphs a more holistic view and facilitates complex queries, analytics, and insights.

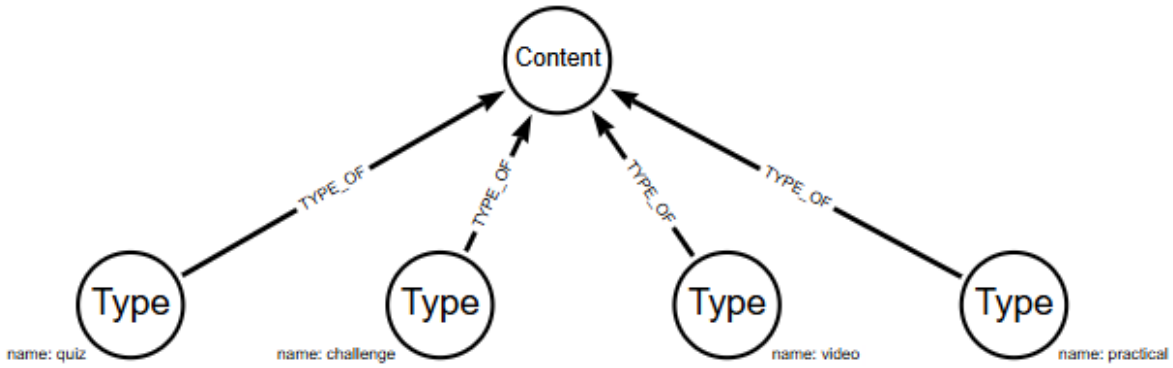
Knowledge graphs can readily adapt and evolve as they grow, taking on new information and structure changes.



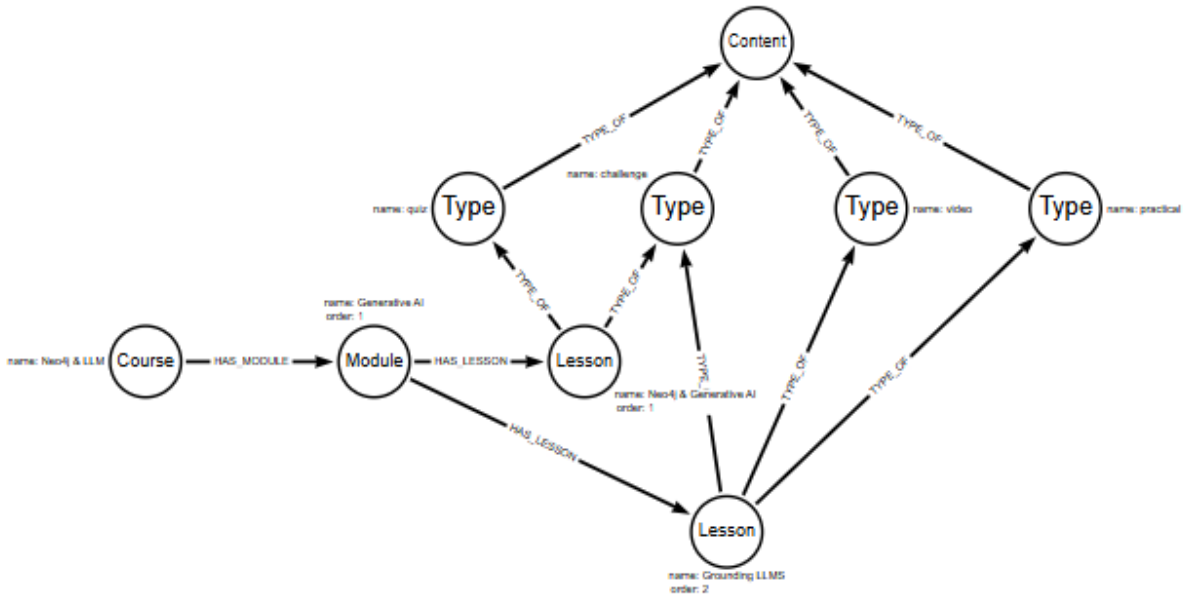
Organizing principles

A knowledge graph stores data and relationships alongside frameworks known as organizing principles.

The organizing principles are the rules or categories around the data that provide structure to the data. Organizing principles can range from simple data descriptions, for example, describing a GraphAcademy course as `course → modules → lessons`, to a complex vocabulary of the complete solution.



The organizing principles are stored as nodes in the graph and can be stored alongside the actual data.



GraphRAG can use knowledge graphs for context, forming the foundation for applications that leverage proprietary or domain-specific data. By grounding responses in a knowledge graph, these applications can provide more accurate answers and greater *explainability*, thanks to the rich context and relationships present in the data.

Creating Knowledge Graphs

Unstructured data

Unstructured data, such as text documents, web pages, or PDFs, can be a rich source for knowledge graphs.

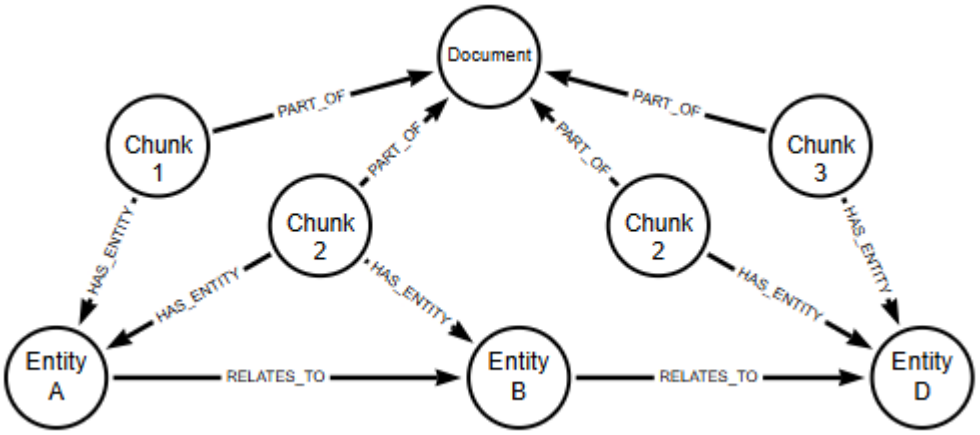
Creating knowledge graphs from unstructured data can be complex, involving multiple steps of data query, cleanse, and transform.

However, you can use the text analysis capabilities of Large Language Models (LLMs) to help automate knowledge graph creation.

Typically, you would follow these steps to construct a knowledge graph from unstructured text using an LLM:

- Gather the data - The data could be from multiple sources and in different formats.
- Chunk the data - Break down the data into manageable parts, or *chunks*, that the LLM can process effectively.
- *Vectorize* the data - Depending on your requirements for querying and searching the data, you may need to create vector embeddings.
- Pass the data to an LLM - Extract entities (nodes) and relationships from the data. You may provide additional context or constraints for the extraction, such as the type of entities or relationships you are interested in extracting.
- Generate the graph - Use the output from the LLM to create nodes and relationships in the graph.

Unstructured Data Structure



Structured data

Constructing knowledge graphs from structured data is often more straightforward than from unstructured sources. Structured data is already organized, making it easier to map nodes, relationships, and relationships directly into a graph.

To create a knowledge graph from structured data, you typically:

1. Identify the sources - The data sources could be other graphs, relational database, CSV files, APIs.
2. Analysize the data - Understand the entities, attributes, and relationships in your data (for example, rows in a table may represent entities, columns as attributes, and foreign keys as relationships).
3. Define a graph schema - The schema should represent the entities as nodes and relationships as well as defining the organizing principles for the graph.
4. Create the graph - Transform and import the data into the graph database.

Introduction to Vector Indexes and Unstructured Data

- Semantic search

1. Tìm kiếm ngữ nghĩa là gì?

Thay vì chỉ khớp các từ khóa chính xác (keyword search) như truyền thống, tìm kiếm ngữ nghĩa tập trung vào việc hiểu **ý định (intent)** và **ngữ cảnh (context)** của người dùng.

- *Ví dụ:* Cùng từ "Apple", hệ thống sẽ phân biệt được bạn đang tìm về "trái táo" hay "tập đoàn công nghệ" dựa trên các yếu tố xung quanh.

2. Cách xác định ngữ cảnh

Hệ thống hiểu ngữ cảnh thông qua 3 yếu tố chính:

- **Thông tin đi kèm:** Các từ khác trong câu truy vấn (ví dụ: "ngân hàng" đi với "dòng sông" sẽ khác với "tài chính").
- **Thông tin người dùng:** Lịch sử tìm kiếm và vị trí địa lý (ví dụ: "football" ở Anh là bóng đá, ở Mỹ là bóng bầu dục).
- **Kịch bản cụ thể:** Tìm kiếm trên một trang web chuyên biệt (ví dụ: tìm từ "dash" trên web về ô tô sẽ hiểu là "bảng điều khiển").

3. Cách xếp hạng kết quả

Kết quả được đánh giá bằng một **điểm số (score)** tổng hợp từ:

- Độ liên quan (relevance).
- Độ phổ biến (popularity).
- Chất lượng của kết quả (quality).

4. Lợi ích và Ứng dụng

Tìm kiếm ngữ nghĩa đặc biệt hữu ích với dữ liệu phi cấu trúc trong các lĩnh vực:

- **Chăm sóc khách hàng:** Giúp Chatbot hiểu và trả lời tự nhiên hơn.
- **Thương mại điện tử:** Gợi ý sản phẩm chính xác theo nhu cầu ẩn ý.
- **Tuyển dụng:** Khớp hồ sơ ứng viên với mô tả công việc dựa trên kỹ năng thực tế thay vì chỉ đếm từ khóa.

- **Quản lý tri thức:** Truy xuất thông tin từ kho dữ liệu lớn hiệu quả hơn.
- **Phát hiện bất thường:** Nhận diện các giao dịch gian lận hoặc tin nhắn lạ.

5. Những thách thức cần lưu ý

5. **Độ phức tạp của ngôn ngữ:** Sự mơ hồ của từ ngữ và các cách diễn đạt khác nhau của mỗi người.
6. **Cần tinh chỉnh (Fine-tuning):** Tốn nhiều công sức để tối ưu hóa mô hình và thuật toán cho từng loại dữ liệu.
7. **Tính minh bạch:** Khó giải thích cụ thể tại sao hệ thống lại đưa ra một kết quả hoặc điểm số nhất định (vấn đề "hộp đen").

8. Vectors

1. Vector là gì?

- **Định nghĩa đơn giản:** Vector là một danh sách các con số (ví dụ: $[1, 2, 3]$).
- **Vai trò:** Dùng để đại diện cho các loại dữ liệu như văn bản, hình ảnh, âm thanh dưới dạng tọa độ trong không gian nhiều chiều.
- **Số chiều (Dimensionality):** Một vector có thể có vài chiều cho đến hàng nghìn chiều (ví dụ: mô hình của OpenAI có 1.536 chiều).

2. Embedding - "Phiên bản số" của ý nghĩa

- **Embedding** là các vector được tạo ra để đại diện cho đối tượng dữ liệu sao cho máy tính có thể so sánh chúng.
- **Đặc điểm:** Mỗi chiều trong vector đại diện cho một khía cạnh ngữ nghĩa.
    - *Ví dụ:* Từ "apple" sẽ có các chiều giá trị cho "trái cây", "công nghệ", "màu sắc", "vị giác"... Khi kết hợp lại, chúng tạo nên ý nghĩa tổng thể.
- **Ngữ cảnh:** Embedding giúp phân biệt các từ giống nhau nhưng nghĩa khác nhau (như "bank" trong "river bank" và "savings bank") dựa vào các từ xung quanh.

3. Cách Vector hoạt động trong tìm kiếm ngữ nghĩa

Nguyên lý cốt lõi là đo lường **khoảng cách** hoặc **góc** giữa các vector:

- **Tương đồng:** Các từ có ý nghĩa hoặc ngữ cảnh giống nhau sẽ có các vector nằm **gần nhau** trong không gian.
- **Khác biệt:** Các từ không liên quan sẽ nằm **xa nhau**.

Quy trình tìm kiếm:

1. Người dùng nhập câu hỏi (Query).
2. Hệ thống chuyển câu hỏi đó thành một Vector (Embedding).
3. Hệ thống so sánh Vector câu hỏi với Vector của dữ liệu đã có trong kho.
4. Chấm điểm độ tương đồng và trả về kết quả gần nhất.

4. Vector và Neo4j

9. Từ phiên bản **v5.11**, Neo4j đã hỗ trợ **Vector search index**.
10. Điều này cho phép bạn lưu trữ các vector trực tiếp trong đồ thị và truy vấn chúng một cách hiệu quả để tìm ra các nút (nodes) có nội dung tương đồng về mặt ý nghĩa.
11. Unstructured data

1. Dữ liệu phi cấu trúc là gì?

Đây là dạng thông tin **không tuân theo một mô hình hoặc định dạng định sẵn**, khiến việc tổ chức và phân tích trở nên khó khăn.

- **Ví dụ:** Tệp văn bản, email, bài đăng mạng xã hội, video, hình ảnh, tệp âm thanh và trang web.
- **Đặc điểm:** Chứa đựng lượng thông tin rất phong phú nhưng thường đòi hỏi con người hoặc các công cụ phức tạp để diễn giải ý nghĩa.

2. Các thách thức chính

Việc phân tích dữ liệu phi cấu trúc gặp 5 trở ngại lớn:

1. **Thiếu cấu trúc:** Khó sắp xếp và giải thích do không có khuôn mẫu.
2. **Khối lượng lớn:** Dữ liệu dạng này thường cực kỳ đồ sộ, gây quá tải cho các hệ thống xử lý truyền thống.
3. **Sự đa dạng:** Nhiều định dạng khác nhau (văn bản khác với hình ảnh) đòi hỏi các công cụ phân tích riêng biệt.
4. **Chất lượng & Tính nhất quán:** Dễ có sai sót, thông tin nhiễu hoặc không liên quan.
5. **Hiểu biết ngữ cảnh:** Cần sự phân tích chuyên sâu để nắm bắt ý nghĩa thực sự đằng sau dữ liệu.

3. Giải pháp: Sự kết hợp giữa Vector và Đồ thị (Graphs)

Để vượt qua các thách thức trên, chúng ta sử dụng hai công cụ bổ trợ cho nhau:

- **Vectors & Embeddings:** Biến dữ liệu phi cấu trúc thành các con số để máy tính có thể **so sánh độ tương đồng** và tìm kiếm thông tin liên quan nhanh chóng.
- **Đồ thị (Graphs):** Giúp **trực quan hóa và phân tích các mối quan hệ**, kết nối giữa các điểm dữ liệu khác nhau.

Ví dụ thực tế:

- Dùng **Vector** để tìm đúng tài liệu hướng dẫn khớp với câu hỏi của khách hàng.
- Dùng **Đồ thị** để hiểu mối liên hệ giữa các sản phẩm cụ thể và những phản hồi (feedback) mà khách hàng để lại.

Vector indexes

- What embeddings are and how to load them into Neo4j

1. Các loại mô hình Embedding phổ biến

Mỗi loại dữ liệu (văn bản, hình ảnh, đồ thị) sẽ có những mô hình chuyên biệt để chuyển đổi chúng thành vector:

Loại dữ liệu	Mô hình tiêu biểu	Đặc điểm chính
Văn bản (Từ/Câu)	**Word2Vec**, **FastText**, **GPT-4**	Chuyển từ hoặc câu thành vector dựa trên ngữ cảnh xung quanh.
Văn bản (Đoạn/Hồ sơ)	**Doc2Vec**, **Universal Sentence Encoder**	Nắm bắt ý nghĩa tổng thể của cả một đoạn văn hoặc tài liệu dài.
Hình ảnh	**ResNet**, **VGGNet**	Trích xuất các đặc điểm thị giác (màu sắc, hình dạng, hoa văn) thành vector.
Đồ thị (Graph)	**Node2Vec**	Tạo embedding dựa trên các bước đi ngẫu nhiên (random walks) giữa các nút trong sơ đồ.

2. Quy tắc quan trọng: "Tính đồng nhất"

Đây là lưu ý kỹ thuật quan trọng nhất khi làm việc với Vector:

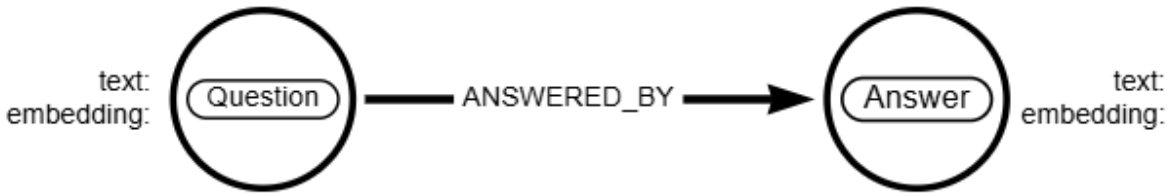
- **Không thể so sánh chéo:** Bạn không thể so sánh một vector tạo bởi mô hình OpenAI với một vector tạo bởi mô hình Google (hoặc kể cả hai phiên bản khác nhau của cùng một hãng).
- **Lý do:** Mỗi mô hình "nhìn" dữ liệu và sắp xếp chúng vào không gian số theo những quy luật khác nhau. Để so sánh độ tương đồng, **bạn phải dùng duy nhất một mô hình** cho cả kho dữ liệu và câu truy vấn.

3. Cách tạo Embeddings

- **Sử dụng mô hình có sẵn (Pre-trained):** Thay vì tự huấn luyện (rất tốn kém), hầu hết các nhà phát triển sử dụng các mô hình đã được huấn luyện trên kho dữ liệu khổng lồ.
- **Sử dụng API:** Các nhà cung cấp (như OpenAI) cung cấp các cổng kết nối (API). Bạn chỉ cần gửi đoạn văn bản lên, và họ sẽ trả về mảng các con số (vector).

- How to create and query vector indexes in Neo4j

Load into Neo4j



Review the following Cypher statement to load the data into Neo4j and create the nodes and relationships:

```
LOAD CSV WITH HEADERS

FROM 'https://data.neo4j.com/llm-vectors-unstructured/Quora-QuAD-1000-embeddings.csv' AS row


MERGE (q:Question{text:row.question})

WITH row,q

CALL db.create.setNodeVectorProperty(q, 'embedding',
apoc.convert.fromJsonList(row.question_embedding))


MERGE (a:Answer{text:row.answer})

WITH row,a,q

CALL db.create.setNodeVectorProperty(a, 'embedding',
apoc.convert.fromJsonList(row.answer_embedding))
```

```
MERGE(q)-[:ANSWERED_BY]->(a)
```

You should be able to identify:

- That the file is loaded using the `LOAD CSV` command.
- The `Question` and `Answer` nodes are created using the `MERGE` command.
- The `embedding` property is set using the `setNodeVectorProperty` function.
- The `apoc.convert.fromJsonList` function converts the embedding string to a list of numbers.
- The `ANSWERED_BY` relationship is created between the `Question` and `Answer` nodes.

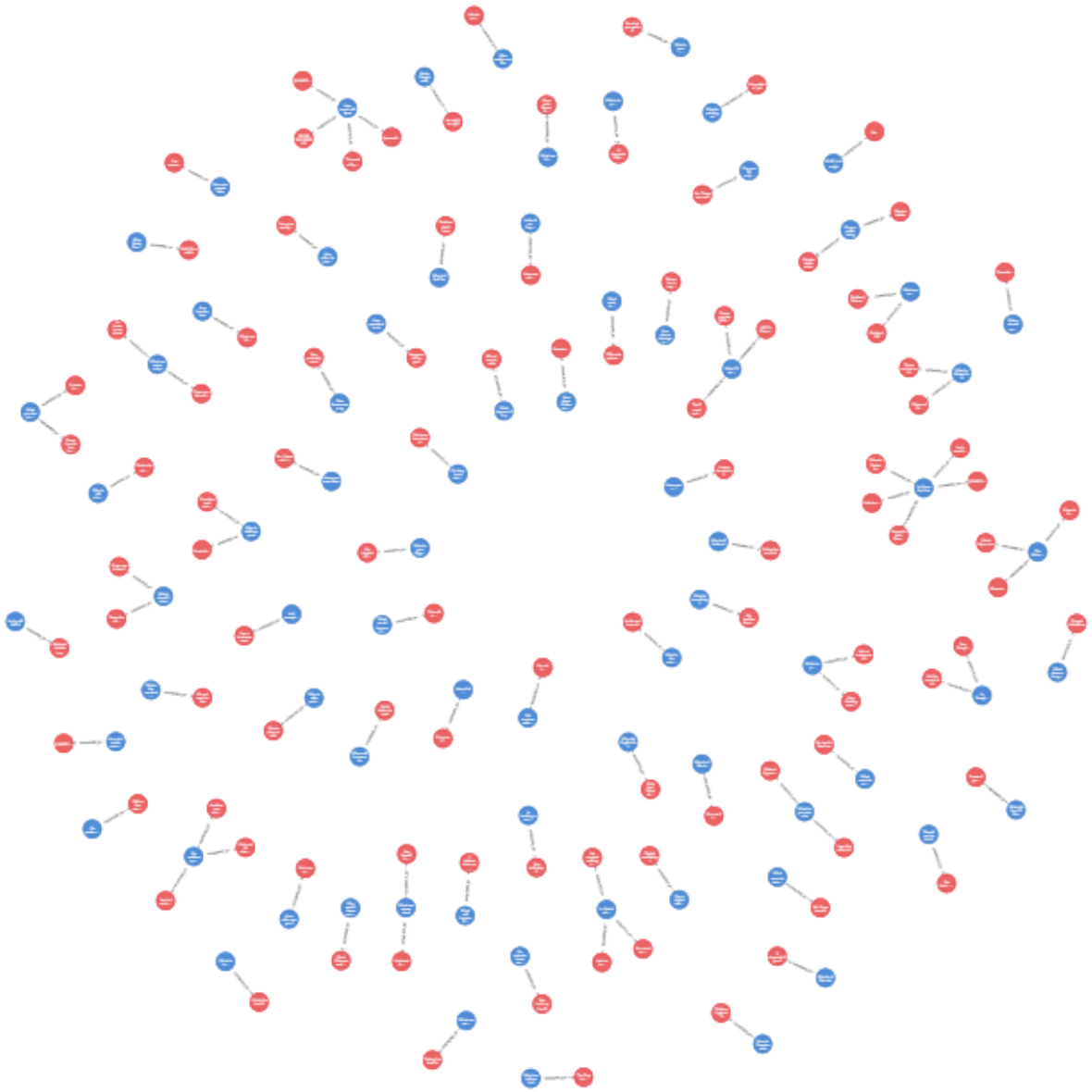
You can check the data was loaded correctly by viewing the `Question` and `Answer` nodes:

```
MATCH (q:Question)-[r:ANSWERED_BY]->(a:Answer)

RETURN q,r,a

LIMIT 100
```

You should see `Question` and `Answer` node connected with the `ANSWERED_BY` relationship.



Create a Vector Index

Create the Question Index

You will use the `CREATE VECTOR INDEX` Cypher statement to create the index:

```
CREATE VECTOR INDEX [index_name] [IF NOT EXISTS]
FOR (n:LabelName)
ON (n.propertyName)
```

```
OPTIONS "{" option: value[, ...] "}"
```

CREATE VECTOR INDEX expects the following parameters:

- index_name - The name of the index
- LabelName - The node label on which to index
- propertyName - The property on which to index
- OPTIONS - The options for the index, where you can specify:
    - vector.dimensions - The dimension of the embedding e.g. OpenAI embeddings consist of 1536 dimensions.
    - vector.similarity_function - The similarity function to use when comparing values in this index - this can be euclidean or cosine.

Check the index creation status

The index will be updated asynchronously. You can check the status of the index population using the SHOW INDEXES statement:

Check that you created the index successfully using the SHOW INDEXES command.

```
SHOW INDEXES WHERE type = "VECTOR"
```

You should see a result similar to the following:

id	name	state	populationPercent	type
1	"questions"	"ONLINE"	100.0	"VECTOR"

Once the state is listed as online, the index will be ready to query.

The populationPercentage field indicates the proportion of node and property pairing.

When the populationPercentage is 100.0, all the question embeddings have been indexed.

Querying with Embeddings

You are going to explore two scenarios:

1. A user views an existing question and wants to see similar questions.
2. A user submits a new question and receives answers to a similar question.

In the first scenario, you will use existing question embeddings to find similar questions. In the second scenario, you will generate a new embedding for the user's question to find similar questions and answers.

Finding similar questions

```
MATCH (q:Question {text: "What are the most touristic countries in the world?"})

CALL db.index.vector.queryNodes('questions', 6, q.embedding)

YIELD node, score

RETURN node.text, score
```

Breaking down the query, you can identify the following:

- The `MATCH` clause finds the specific `Question` node.
- The query uses the `db.index.vector.queryNodes` function to query the `questions` vector index with the `Question` node's embedding - `q.embedding`. The function returns the top `6` similar nodes.
- `YIELD` obtain the `node` and similarity `score` returned by the function.
- The query returns the `Question` node's `text` property and the similarity score.

You can extend this query to return the answers to the most similar questions:

```
MATCH (q:Question {text: "What are the most touristic countries in the world?"})

CALL db.index.vector.queryNodes('questions', 6, q.embedding)

YIELD node, score

MATCH (node)-[:ANSWERED_BY]->(a)

RETURN a.text, score
```

Finding answers to a similar question

You can incorporate the embedding into your query to find similar questions:

```
WITH genai.vector.encode(
  "What are good open source projects",
  "OpenAI",
  { token: "sk-..." }) AS userEmbedding
CALL db.index.vector.queryNodes('questions', 6, userEmbedding)
YIELD node, score
RETURN node.text, score
```

This query, creates an embedding using `genai.vector.encode` and then uses that embedding to query the `questions` vector index.

- The different similarity functions available in Neo4j

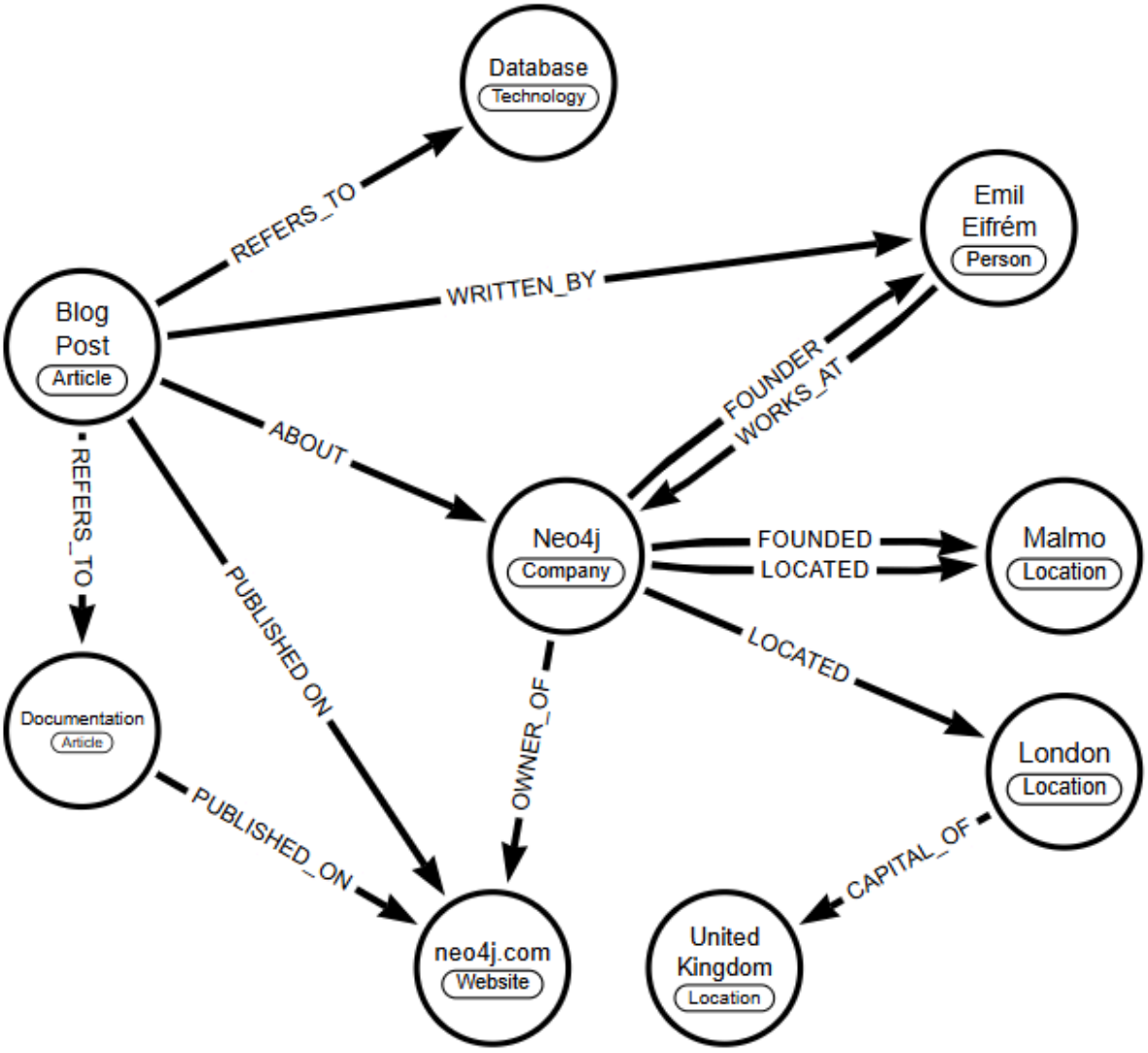
Hàm Similarity	**Cách hoạt động**	**Trường hợp sử dụng tốt nhất**
Cosine	Đo góc giữa 2 vector. Không quan tâm văn bản dài hay ngắn.	**Phổ biến nhất cho Văn bản (NLP)**. Phù hợp nhất cho Chatbot và tìm kiếm ngữ nghĩa.
Euclidean (L2)	Đo khoảng cách đường thẳng giữa 2 điểm.	Thường dùng cho dữ liệu hình ảnh hoặc các phép đo vật lý chính xác.
Dot Product	Tích vô hướng (kết hợp cả hướng và độ dài).	Dùng khi tần suất xuất hiện của từ (độ dài vector) mang thông tin quan trọng.

Knowledge graphs

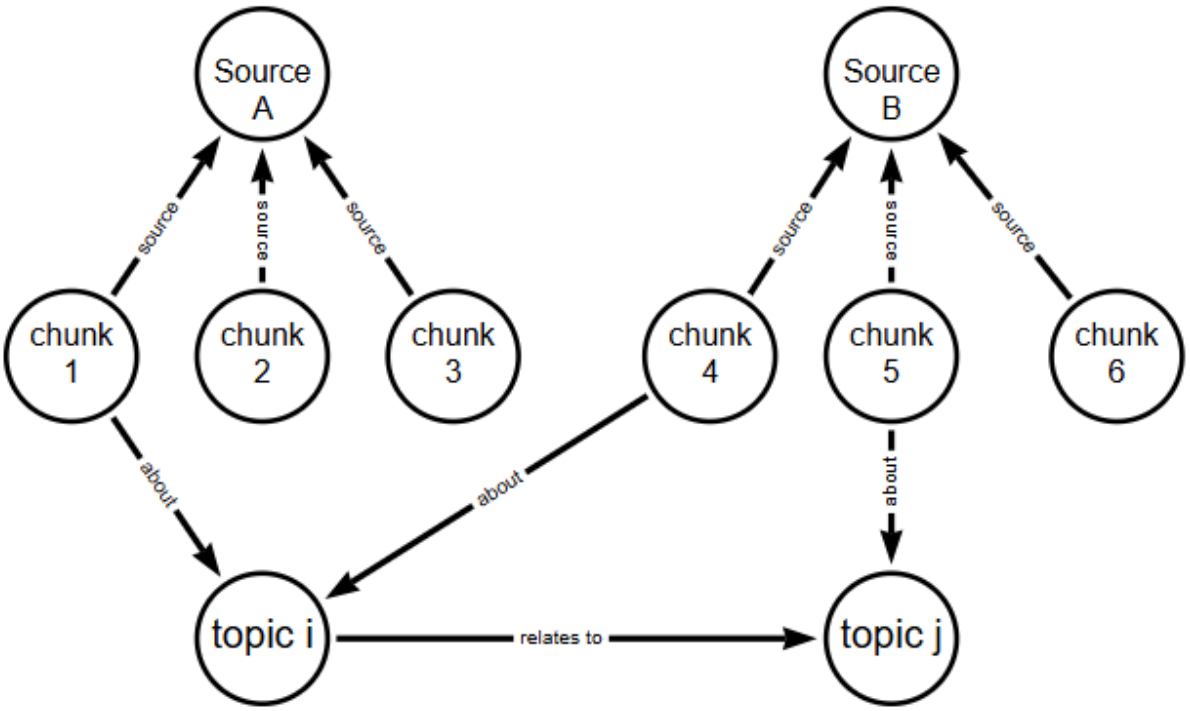
- What a Knowledge Graph is.

A knowledge graph is an organized representation of real-world entities and their relationships.

Knowledge graphs provide a structured way to represent entities, their attributes, and their relationships, allowing for a comprehensive and interconnected understanding of the information.



Knowledge graphs can break down sources of information and integrate them, allowing you to see the relationships between the data.



This integration from diverse sources gives knowledge graphs a more holistic view and facilitates complex queries, analytics, and insights.

Knowledge graphs can readily adapt and evolve as they grow, taking on new information and structure changes.

- The benefits and challenges of creating and using a Knowledge Graph.

Benefits of knowledge graphs:

- Enhanced Data Integration and Interoperability
  Knowledge graphs facilitate data integration from disparate sources, creating a unified view.
- Improved Search and Discovery
  Knowledge graphs enhance search capabilities by structuring data into entities and relationships. Instead of simple keyword-based searches, users can perform more nuanced queries that reflect real-world contexts and relationships.
- Contextual Understanding
  Knowledge graphs help understand the context of information by capturing the relationships between entities.
- Enhanced Decision Making
  Knowledge graphs enable better decision-making by providing a holistic view of

interconnected data. Organizations can identify patterns and trends that might not be apparent from isolated data points.

Challenges

While the benefits of knowledge graphs are substantial, creating and maintaining them comes with challenges. These challenges span from data acquisition and integration, including:

- Data Collection and Integration
  Data is often spread across various sources with different formats, structures, and standards.
- Data Quality
  Ensuring data's accuracy, consistency, and completeness is crucial.
- Data Modeling & Schema Design
  Developing a flexible and robust schema that can accommodate a wide range of entities and relationships is challenging. The schema must be adaptable to evolving data requirements and scalable to handle large datasets.
- Entity Resolution
  Identifying and merging duplicate entities from different sources (e.g., different representations of the same person or company).
- Entity Linking & Relationships
  Accurately identifying and correctly associating entities with their corresponding entries in the knowledge graph is crucial for accuracy and understanding relationships.

- About typical Knowledge Graph use case

1. Hợp nhất và Kết nối dữ liệu (Data Integration)

Đồ thị tri thức đóng vai trò như một "bộ khung" chung để gắn kết các nguồn dữ liệu rời rạc lại với nhau.

- **Cách hoạt động:** Tạo ra một cấu trúc thống nhất giữa các thực thể và mối quan hệ.
- **Lợi ích:** Giúp các hệ thống khác nhau có thể "nói chuyện" và trao đổi dữ liệu với nhau một cách liền mạch (tính tương tác cao).

2. Cải thiện Tìm kiếm và Truy xuất thông tin

Thay vì chỉ tìm kiếm dựa trên từ khóa đơn thuần, đồ thị tri thức hiểu được ý nghĩa của dữ liệu.

- **Cách hoạt động:** Tận dụng các mối quan hệ ngữ nghĩa giữa các điểm dữ liệu.

- **Lợi ích:** Kết quả tìm kiếm chính xác hơn, hiểu được ngữ cảnh của người dùng, giúp tìm đúng thông tin cần thiết nhanh hơn.

3. Phân tích nâng cao và Khám phá tri thức (Analytics & Insights)

Đồ thị cho phép thực hiện các truy vấn phức tạp trên dữ liệu có tính kết nối cao.

- **Cách hoạt động:** Thực hiện suy luận (reasoning) trên các dữ liệu liên kết để tìm ra các mối quan hệ ẩn.
- **Lợi ích:** Khám phá ra các xu hướng, quy luật ngầm mà các cơ sở dữ liệu truyền thống không thấy được, từ đó hỗ trợ ra quyết định chính xác và dự báo tương lai.

4. Cá nhân hóa trải nghiệm người dùng

Đồ thị tri thức có thể lưu trữ "hồ sơ" chi tiết về mối quan tâm của người dùng.

- **Cách hoạt động:** Ghi lại sở thích, hành vi và ngữ cảnh của từng cá nhân trong đồ thị.
- **Lợi ích:** Đưa ra các gợi ý sản phẩm, nội dung hoặc dịch vụ "đo ni đóng giày" cho từng người, giúp tăng mức độ hài lòng và gắn bó của khách hàng.

How to Construct a Knowledge Graph with an LLM

The construction process

Typically, you would follow these steps to construct a knowledge graph from unstructured text using an LLM:

1. Gather the data
2. Chunk the data
3. *Vectorize* the data
4. Pass the data to an LLM to extract nodes and relationships
5. Use the output to generate the graph

Gather your data sources

The first step is to gather your unstructured data. The data can be in the form of text documents, PDFs, publicly available data, or any other source of information.

Depending on the format, you may need to reformat the data into a format (typically text) that the LLM can process.

The data sources should contain the information you want to include in your knowledge graph.

Chunk the data

The next step is to break down the data into *right-sized* parts. This process is known as *chunking*.

The size of the chunks depends on the LLM you are using, the complexity of the data, and what you want to extract from the data.

You may not need to chunk the data if the LLM can process the entire document at once and it fits your requirements.

Vectorize the data

Depending on your requirements for querying and searching the data, you may need to create vector embeddings. You can use any embedding model to create embeddings for each data chunk, but the same model must be used for all embeddings.

Placing these vectors into a Vector index allows you to perform semantic searches, similarity searches, and clustering on the data.

Extract nodes and relationships

The next step is to pass the unstructured text data to the LLM to extract the nodes and relationships.

You should provide a suitable prompt that will instruct the LLM to:

- Identify the entities in the text.
- Extract the relationships between the entities.
- Format the output so you can use it to generate the graph, for example, as JSON or another structured format.

Optionally, you may also provide additional context or constraints for the extraction, such as the type of entities or relationships you are interested in extracting.

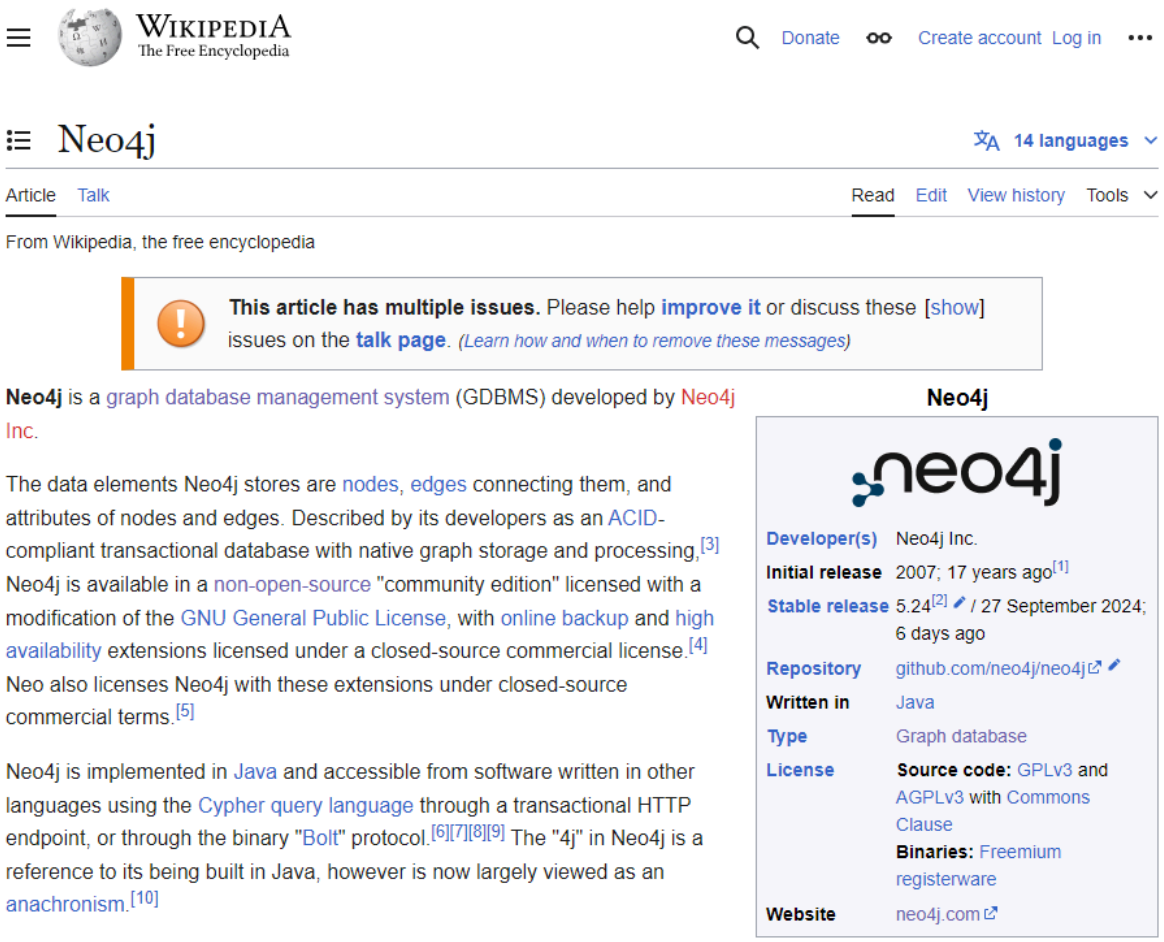
Generate the graph

Finally, you can use the output from the LLM to generate the graph by creating the nodes and relationships within Neo4j.

The entity and relationship types would become labels and relationship types in the graph. The *names* would be the node and relationship identifiers.

Example

If you wanted to construct a knowledge graph based on the Neo4j Wikipedia page, you would:



1. Gather the text from the page.

Neo4j is a graph database management system (GDBMS) developed by

Neo4j Inc.
The data elements Neo4j stores are nodes, edges connecting them

and attributes of nodes and edges. Described by its developers

as an ACID-compliant transactional database with native graph

storage and processing...

2. Split the text into chunks.
Neo4j is a graph database management system (GDBMS) developed

by Neo4j Inc.
The data elements Neo4j stores are nodes, edges connecting them

and attributes of nodes and edges.
Described by its developers as an ACID-compliant transactional

database with native graph storage and processing…

3.Generate embeddings and vectors for each chunk.
[0.21972137987, 0.12345678901, 0.98765432109, ...]
[0.34567890123, 0.23456789012, 0.87654321098, ...]
[0.45678901234, 0.34567890123, 0.76543210987, ...]

4. Extract the entities and relationships using an LLM.

Send the text to the LLM with an appropriate prompt, for example:

Your task is to identify the entities and relations requested

with the user prompt from a given text. You must generate the

output in a JSON format containing a list with JSON objects.

Text:

{text}

Parse the entities and relationships output by the LLM.


```
[
    {
        "text": (
            "Neo4j is a graph database management system (GDBMS) developed by Neo4j Inc."
        ),
        "head": "Neo4j",
        "head_type": "GraphDatabase",
        "relation": "DEVELOPED_BY",
        "tail": "Neo4j Inc",
        "tail_type": "Company",
    },
    {
        "text": (
            "Neo4j is implemented in Java"
        ),
        "head": "Neo4j",
        "head_type": "Graph Database",
        "relation": "IMPLEMENTED_IN",
        "tail": "Java",
        "tail_type": "ProgrammingLanguage",
    },
    ...
]
```

5. Generate the graph.

Use the data to construct the graph in Neo4j by creating nodes and relationships based on the entities and relationships extracted by the LLM.

```
MERGE (neo4jInc:Company {id: 'Neo4j Inc'})
MERGE (neo4j:GraphDatabase {id: 'Neo4j'})
MERGE (java:ProgrammingLanguage {id: 'Java'})
MERGE (neo4j)-[:DEVELOPED_BY]->(neo4jInc)
MERGE (neo4j)-[:IMPLEMENTED_IN]->(java)
```